GROUP PROJECT CONTRIBUTION TRACKER

A PROJECT REPORT

Submitted by

MITHRAN VT

(240701312)

in partial fulfilment of the End Semester Examination for the course

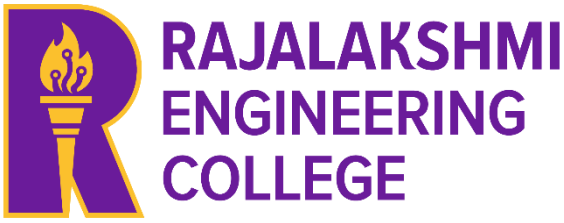
CB23F35

LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF ENGINEERING

COMPUTER SCIENCE ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

NOVEMBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled "**GROUP PROJECT CONTRIBUTION TRACKER**" is the bonafide work of MITHRAN VT(240701312) who carried out the individual project work on the Zoho Creator low-code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator, during the academic year [2026–2027], Semester [V], under my supervision.

SIGNATURE

DR MANORANJINI
HEAD OF THE DEPARTMENT
Department of CSE
Rajalakshmi Engineering College
Thandalam – 602 105

SIGNATURE

DR JINU SOPHIA J
COURSE FACULTY
Department of CSE
Rajalakshmi Engineering College

Submitted for the End Semester Examination (Individual Project) held on [ ].

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

I, MITHRAN VT (240701312), hereby declare that the project report titled “GROUP PROJECT CONTRIBUTION TRACKER”, submitted for the End Semester Examination of the course CB23F35 – Low-Code Application Development using Zoho Creator, is a record of original work carried out by me individually. The Zoho Creator application described in this report — including its data model, forms, reports, pages, Deluge scripts, workflows and integrations — was designed and developed by me.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit. Every template, sample application, AI-assisted feature (such as Zia or the AI application builder), code snippet or external resource used in this work has been duly acknowledged in the report.

Place: Chennai
Date:

SIGNATURE OF THE STUDENT
MITHRAN VT
240701312

ACKNOWLEDGEMENT

I express my sincere gratitude to the management of Rajalakshmi Engineering College for providing the facilities and learning environment required to complete this project.

I thank the Principal of Rajalakshmi Engineering College for encouraging students to undertake practical and industry-oriented projects.

I thank Dr. Manoranjini, Head of the Department, Department of Computer Science and Engineering, for the support provided during the course.

I express my sincere thanks to Jinu Sophia J, Course Faculty, for the guidance and feedback provided for this project.

I also acknowledge Zoho Creator and its learning resources for providing a platform to explore low-code application development. I thank my family and friends for their encouragement.

MITHRAN VT

ABSTRACT

Group projects are a common part of academic learning in which students work together to complete tasks, prepare deliverables and meet deadlines. In many student teams, project planning and contribution records are maintained through chat messages, spreadsheets or verbal updates. These methods may be sufficient for small tasks, but they become difficult to manage when several members are responsible for different activities. Information about assigned tasks, progress, effort and individual participation may be scattered across multiple places. The Group Project Contribution Tracker is proposed as an academic productivity application for student teams and faculty coordinators. It provides a central place to record projects, team members, tasks and contribution logs.

The proposed Group Project Contribution Tracker is designed using Zoho Creator. Its main forms are Project, Team Member, Task and Contribution Log, with optional Peer Feedback records. A project can be linked to multiple members and tasks, and contribution entries can record the work completed, date, effort hours and evidence or notes. Reports can show tasks by status, upcoming deadlines, project progress and contribution summaries. Deluge scripts and form workflows may be used to validate entries, update statuses and send notifications where configured. Role-based access can distinguish an administrator or project coordinator from student members. The design uses Zoho Creator's built-in forms, reports, pages and workflow capabilities rather than requiring a separately developed database and web server.

The application should be evaluated by entering sample projects, assigning tasks, recording contributions and checking whether reports reflect the saved data correctly. Testing should include required-field validation, date and status handling, duplicate or incomplete entries, and permissions for different roles. Since deployment details, screenshots, test results and performance measurements depend on the actual Zoho Creator build, these must be added after the application is tested; no numerical improvement is claimed in this draft. The intended benefit is to reduce scattered records, improve visibility of team progress and make contribution discussions more evidence-based. Keywords: Low-Code Development; Zoho Creator; Group Projects; Contribution Tracking; Deluge Scripting; Workflow Automation.

Keywords: Low-Code Development; Zoho Creator; Group Projects; Contribution Tracking; Deluge Scripting; Workflow Automation

PROJECT AT A GLANCE

Attribute	Details
Project Title	Group Project Contribution Tracker
Application Domain	Education / Academic Project Management
Student Name & Register No.	MITHRAN VT – 240701312
Zoho Creator Edition	Trial account
Application Name (Link Name)	Group_Project_Contribution_Tracker
Published / Live URL	NA
Demo Credentials (per role)	NA
Number of Forms / Subforms	23
Number of Reports (by type)	3
Number of Pages / Dashboards	4
Number of Workflows	7
Deluge Functions / Custom Actions	Enter actual count after implementation
Integrations	None
Roles & Profiles Defined	Proposed: Administrator/Coordinator and Student Member
Environments Used	N/A
Date of Production Deployment	N/A

TABLE OF CONTENTS

Formatting Standards (Anna University Convention) ..... **Error! Bookmark not defined.**

Working with the Template ..... **Error! Bookmark not defined.**

ABSTRACT iv

PROJECT AT A GLANCE vi

LIST OF TABLES x

LIST OF FIGURES ..... **Error! Bookmark not defined.**

LIST OF ABBREVIATIONS xii

INTRODUCTION 1

1.1 Background 1

1.2 Problem Statement 1

1.3 Objectives 1

1.4 Scope of the Project 2

1.5 Justification for a Low-Code Approach 2

1.6 Organisation of the Report 2

LITERATURE SURVEY AND SYSTEM STUDY 4

2.1 Study of the Existing System 4

2.2 Limitations of the Existing System 4

2.3 Low-Code versus Traditional Development 5

2.4 Literature Survey 5

2.5 Emerging Trends in Low-Code Development 6

2.6 Proposed System 6

REQUIREMENT ANALYSIS 8

3.1 Stakeholders and User Roles 8

3.2 Functional Requirements 8

3.3 Non-Functional Requirements 10

3.4 Hardware and Software Requirements 10

3.5 Use Case Model 10

SYSTEM DESIGN 12

4.1 System Architecture 12

4.2 Data Model 12

4.3 Form Design 14

4.4 Process and Workflow Design 14

4.5 User Interface and Navigation Design 15

4.6 Access Control Design 15

IMPLEMENTATION 16
5.1 Application Setup 16
5.2 Forms and Data Capture 16
5.3 Reports 17
5.4 Pages and Dashboards 18
5.5 Deluge Scripting 19
5.6 Workflows and Automation 20
5.7 Integrations 22
5.8 User Management and Role-Based Access 22
5.9 AI and Analytics Features 23
5.10 Pro-Code Extension 23
5.11 Mobile and Portal Access ..... **Error! Bookmark not defined.**
TESTING AND DEBUGGING 24
6.1 Testing Strategy 24
6.2 Test Cases 24
6.3 Debugging Deluge Scripts 25
6.4 Defect Log 26
6.5 User Acceptance Testing 26
DEPLOYMENT, SECURITY AND GOVERNANCE 27
7.1 Environments and Deployment Strategy 27
7.2 Performance Optimisation 27
7.3 Security Features 28
7.4 Governance and Administration 28
RESULTS AND DISCUSSION 30
8.1 Application Output 30
8.2 Achievement of Objectives 31
8.3 Comparison with the Existing System 32
8.4 Limitations of the Low-Code Approach 32
8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum 33
CONCLUSION AND FUTURE ENHANCEMENTS 34
9.1 Conclusion 34
9.2 Future Enhancements 34
9.3 Learning Outcomes and Career Relevance 34
REFERENCES 35
APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST 36
APPENDIX B – COURSE OUTCOME MAPPING 39
APPENDIX C – COMPLETE DELUGE SOURCE CODE 40

APPENDIX D – USER MANUAL 42
D.1 Accessing the Application 42
D.2 Instructions for [Role 1] 42
D.3 Instructions for [Role 2] 42
APPENDIX E – EVALUATION RUBRIC 44
APPENDIX F – VIVA-VOCE RECORD SHEET 46

LIST OF TABLES

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application....5
Table 2.2: Summary of Literature Survey....5
Table 3.1: Stakeholders and User Roles....8
Table 3.2: Functional Requirements....8
Table 3.3: Non-Functional Requirements....10
Table 3.4: Hardware and Software Requirements....10
Table 3.5: Use Case Description – [Use Case Name]. **Error! Bookmark not defined.**
Table 4.1: Entities and Relationships....13
Table 4.2: Field Specification – [Form Name] Form.. **Error! Bookmark not defined.**
Table 4.3: Role–Permission Matrix....15
Table 5.1: Reports Implemented....17
Table 5.2: Deluge Scripts Implemented....19
Table 5.3: Workflows Implemented....20
Table 5.4: Integrations Implemented....22
Table 6.1: Test Cases....24
Table 6.2: Defect Log....26
Table 6.3: User Acceptance Feedback....26
Table 7.1: Deployment Checklist....27
Table 7.2: Performance Optimisation Results....28
Table 7.3: Security Controls Implemented....28
Table 8.1: Mapping of Objectives to Outcomes....31
Table 8.2: Existing System versus Proposed Application....32
Table A.1: Feature Coverage Checklist....36
Table B.1: Mapping of Course Outcomes to Report Chapters....39
Table E.1: Evaluation Rubric for the Individual Project....44
Table E.2: Performance Level Descriptors....45
Table F.1: Viva-Voce Questions and Responses....46
Table F.2: Marks Summary....46

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
ER	Entity–Relationship
JSON	JavaScript Object Notation
LCAP	Low-Code Application Platform
OAuth	Open Authorization
RBAC	Role-Based Access Control
REST	Representational State Transfer
SDLC	Software Development Life Cycle
SSO	Single Sign-On
UAT	User Acceptance Testing
UI	User Interface
Zia	Zoho’s AI assistant

CHAPTER 1
INTRODUCTION

1.1 Background

The project belongs to the education and academic project-management domain. Student teams often divide a larger assignment into tasks such as research, design, coding, testing, documentation and presentation preparation. In an informal process, the team leader may assign work through messages and track progress using a spreadsheet. Each member reports progress differently, and records of completed work or time spent may not be maintained consistently. This project models a small academic team environment rather than a specific commercial organisation. It aims to support student members and a project coordinator by keeping project details, task assignments, deadlines and contribution records in one structured application.

1.2 Problem Statement

Student group projects require coordination of tasks, deadlines and individual contributions, but informal tracking through chats, spreadsheets and verbal updates can result in incomplete records and unclear accountability. The team may not have a single view of who is responsible for each task, which activities are delayed or what work has been completed. The Group Project Contribution Tracker addresses this problem by providing a centralised, searchable record of projects, members, assigned tasks and contribution updates.

- Project information and task updates may be spread across chats, documents and separate spreadsheets.
- Task ownership, due dates and progress may not be visible to every team member in one place.
- Individual contributions may be difficult to review consistently when there is no structured activity log or evidence field.

1.3 Objectives

- To design a structured data model and forms for projects, team members, tasks and contribution logs.
- To use form validation and Deluge workflows to support accurate task and contribution entry.

- To provide reports and a dashboard showing task status, deadlines and contribution summaries.
- To configure role-based access and, if available and actually implemented, notifications through a Zoho service.
- To test the application with realistic sample data and document deployment, access control and governance settings.

1.4 Scope of the Project

In scope are project registration, member details, task assignment, due dates, status tracking, contribution logs, summary reports and role-based access. The prototype is intended for academic group work and can be demonstrated with sample records. Out of scope for the initial version are automatic grading, biometric attendance, plagiarism detection, payroll, and guaranteed measurement of a student's actual effort. Contribution hours and notes are self-reported and should be reviewed by the team or faculty coordinator. External integrations and production deployment are included only if they are actually configured.

1.5 Justification for a Low-Code Approach

Zoho Creator is appropriate because the application mainly requires structured forms, relationships between records, reports, simple business rules and access permissions. Its visual form and report builders reduce the amount of code required for a working prototype, while Deluge can handle validation and workflow logic when configuration alone is insufficient. The platform also provides hosted access and mobile support, subject to the selected edition and account settings. This lets a student focus on the data model and workflow instead of separately building a web server, database and authentication layer. Zoho Creator documentation should be consulted for the exact capabilities available in the edition used [1].

1.6 Organisation of the Report

Chapter 2 reviews the existing system and related low-code literature. Chapter 3 presents the requirement analysis. Chapter 4 describes the system design. Chapter 5 details the implementation on Zoho Creator. Chapter 6 covers testing and debugging. Chapter 7 discusses deployment, performance optimisation, security and governance.

Chapter 8 presents results and discussion, and Chapter 9 concludes the report with future enhancements.

CHAPTER 2
LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

A typical existing process starts when the team receives an assignment. The team leader creates a chat group, discusses the scope and allocates tasks in messages or a spreadsheet. Members send progress updates informally and share files through separate tools. The leader manually checks due dates and asks members for status updates. At the end, the team attempts to summarise who completed each activity, often using memory and scattered evidence. This process has no single authoritative record linking each task to its owner, status, due date and contribution history.

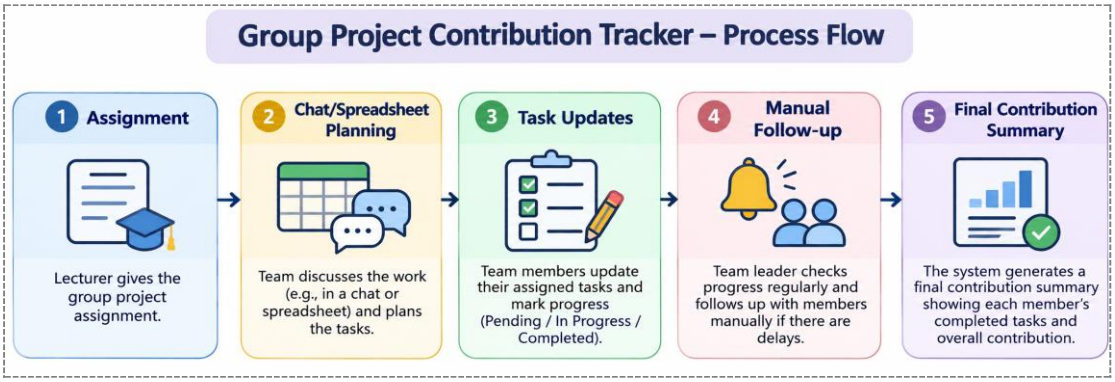


Figure 2.1: Existing Process Flow — Assignment → Chat/Spreadsheet Planning → Task Updates → Manual Follow-up → Final Contribution Summary

2.2 Limitations of the Existing System

- Information is duplicated or becomes outdated when the same project details are maintained in multiple places.
- There is no consistent, real-time overview of task status, ownership and approaching deadlines.
- Contribution summaries rely on manual compilation and may not include a clear history of work entries.

2.3 Low-Code versus Traditional Development

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application

Criterion	Traditional (Pro-Code) Development	Low-Code on Zoho Creator
Development time	Requires coding UI, database, authentication and hosting; effort depends on scope	Visual forms and reports can shorten initial prototype development
Skills required	Programming, database, web framework and deployment skills	Form design, data modelling and basic Deluge/workflow knowledge
Hosting & infrastructure	Developer must select and maintain hosting/database	Hosting is provided by the platform subject to edition and settings
Mobile application	Usually needs responsive design or separate mobile development	Creator mobile access may be available through the platform
Security & access control	Must be designed and implemented in application code	Built-in user, profile and permission configuration can be used
Maintenance & change	Changes may require code updates and redeployment	Many form/report changes can be made visually
Customisation limits	High flexibility with sufficient development effort	Bound by platform features, Deluge capabilities and plan limits
Cost	Development and hosting costs vary	Subscription/edition costs and platform limits apply

2.4 Literature Survey

Table 2.2: Summary of Literature Survey

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
1	Zoho Corporation, n.d.	Zoho Creator Help Documentation [1]	Documents forms, reports, pages, workflows and application configuration.	Primary reference for building the tracker in Zoho Creator.

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
2	Zoho Corporation, n.d.	Deluge Script Reference Guide [2]	Explains Deluge syntax and supported functions for application logic.	Supports validation and workflow automation where used.
3	Zoho Corporation, n.d.	Zoho Creator Learning Centre [3]	Provides learning material and examples for Creator features.	Useful for learning platform concepts and implementing the prototype.
4	Gartner, use the edition actually consulted	Magic Quadrant for Enterprise Low-Code Application Platforms [4]	Industry analysis discusses low-code platforms and their enterprise capabilities.	Provides broader context for low-code adoption; verify publication details before submission.
5	Add one peer-reviewed paper you actually read	Low-code application development research paper	Summarise the paper's actual findings after reading it.	Use the paper to discuss productivity, maintainability or platform limitations.

2.5 Emerging Trends in Low-Code Development

Low-code development increasingly combines visual application building with configurable automation, reusable components, analytics and AI-assisted creation. For this project, visual forms and reports can be used to create the core application, while Deluge handles rules that need conditional logic. A dashboard can present task counts and contribution summaries. AI-assisted builders may help create an initial draft, but every generated form, formula and script must be checked against the actual requirements. These trends informed the decision to keep the first version modular and simple, while leaving advanced analytics and external integrations as future enhancements.

2.6 Proposed System

The proposed system is a Zoho Creator application that centralises academic project coordination. An administrator or coordinator creates a project, registers team members and assigns tasks with a due date, priority and status. Members add contribution logs

linked to a project and task, describing completed work and optionally recording effort hours or evidence. Reports present tasks grouped by status, overdue items and contributions by member. Form validations help prevent incomplete records, and workflows can notify relevant users when a task is assigned or its status changes if notification capability is configured. The system improves traceability by connecting project, member, task and contribution data.

CHAPTER 3
REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

Table 3.1: Stakeholders and User Roles

Role	Description	Key Responsibilities in the Application
Administrator	Application owner with configuration access	Manage forms, users, permissions and all project records
Project Coordinator	Student team leader or faculty coordinator	Create projects, register members, assign tasks and review contributions
Student Member	A member of a project team	View relevant tasks, update task status and submit contribution logs
Faculty Reviewer (optional)	Faculty member reviewing project progress	View project summaries and contribution reports; provide feedback if module exists

3.2 Functional Requirements

Table 3.2: Functional Requirements

FR ID	Requirement	Role(s)	Priority
FR-01	The system shall allow an authorised user to create a project with title, description, start date and due date.	Administrator, Coordinator	High
FR-02	The system shall allow team members to be registered and associated with a project.	Administrator, Coordinator	High
FR-03	The system shall allow tasks to be assigned to a project and a responsible member with priority, due date and status.	Coordinator	High
FR-04	The system shall allow a member to submit a contribution log linked to a project and task.	Student Member	High
FR-05	The system shall provide reports showing task status, overdue tasks and contribution entries.	Coordinator, Faculty Reviewer	Medium
FR-06	The system shall validate required fields and reject invalid or negative effort-hour values where applicable.	All permitted users	High

FR ID	Requirement	Role(s)	Priority
FR-07	The system should notify relevant users of assignments or updates if notification workflows are configured.	Coordinator, Student Member	Medium

3.3 Non-Functional Requirements

Table 3.3: Non-Functional Requirements

NFR ID	Category	Requirement
NFR-01	Performance	For the expected small academic dataset, standard filtered reports should be usable without unnecessary columns; record actual load times if measured.
NFR-02	Security	Users shall access only records and actions permitted by their assigned role.
NFR-03	Usability	Forms should use clear labels, required-field indicators and consistent status options.
NFR-04	Availability	Application availability depends on the Zoho Creator service, account and internet connectivity.
NFR-05	Maintainability	Forms, reports and scripts should use consistent names and documented logic.
NFR-06	Auditability	Where platform audit/history features are available, changes should be traceable; document the actual configuration.

3.4 Hardware and Software Requirements

Table 3.4: Hardware and Software Requirements

Category	Specification
Platform	Zoho Creator; enter the edition available in your account
Scripting language	Deluge
Browser	Google Chrome / Microsoft Edge / Mozilla Firefox, current supported version
API testing tool	Postman, only if an API integration is tested
Other Zoho services	None assumed; list only services actually connected
Mobile	Zoho Creator mobile app, if tested
Client hardware	Computer or mobile device with a supported browser and internet connection

3.5 Use Case Model

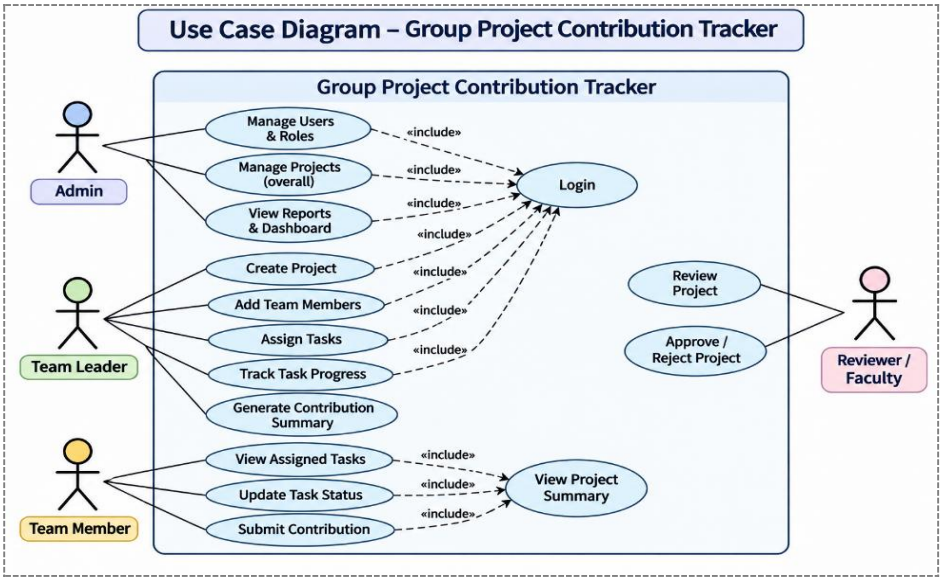


Figure 3.1: Use Case Diagram of the Group Project Contribution Tracker

Table 3.5: Use Case Description – Record a Contribution

Use Case ID	UC-01
Use Case Name	Record a Contribution
Actor(s)	Student Member
Pre-condition	The project and task exist, and the member has permission to add a contribution.
Main Flow	1. Member opens Contribution Log. 2. Selects the project and task. 3. Enters contribution date, description and optional effort hours. 4. Submits the record. 5. The record is saved and appears in the contribution report.
Alternate Flow	If a required field is missing or effort hours are invalid, the form displays a validation message and the record is not saved.
Post-condition	A contribution record is stored and can be reviewed by an authorised coordinator.
Related FR	FR-04, FR-06

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

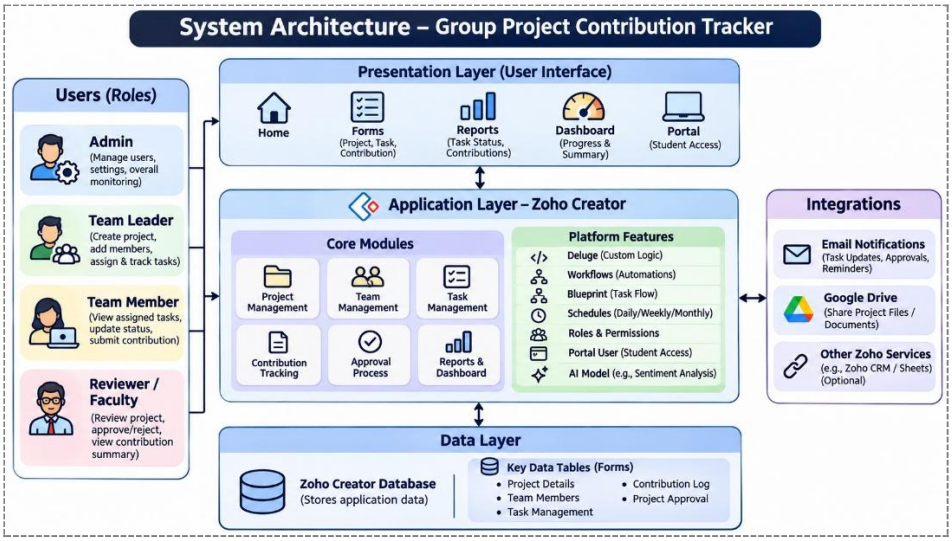


Figure 4.1: System Architecture of the Group Project Contribution Tracker

The system has four logical layers. The presentation layer contains the Zoho Creator web interface and, if used, mobile interface. The application layer contains forms, reports, dashboard pages, form rules and Deluge workflows. The data layer stores project, member, task and contribution records with lookup relationships. The access-control layer defines who can create, view or update each type of record. Optional Zoho Mail or other integrations can be added through authorised connections, but should be documented only after they are configured and tested.

4.2 Data Model

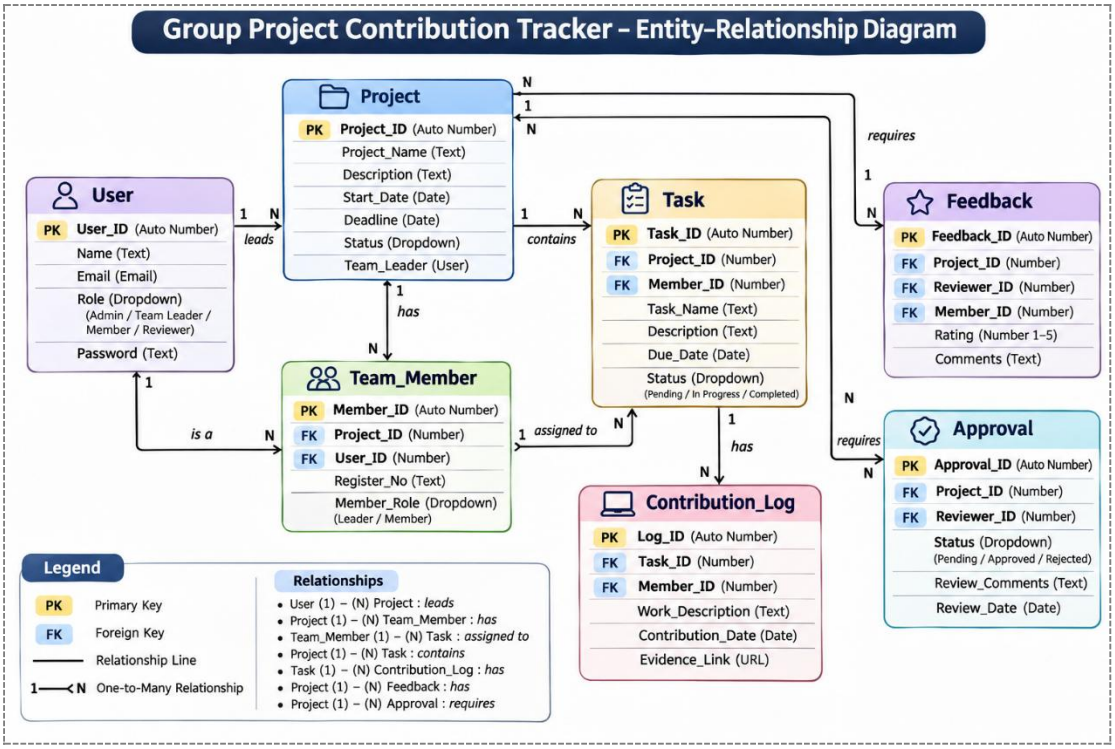


Figure 4.2: Entity–Relationship Diagram

Table 4.1: Entities and Relationships

Entity (Form)	Key Fields	Related To	Relationship Type	Implemented Using
Project	Project_ID, Project_Name, Start_Date, Due_Date, Status	Team Member; Task; Contribution Log	One-to-many	Lookup fields in related forms
Team Member	Member_ID, Member_Name, Email, Role, Project	Project; Task; Contribution Log	Many members belong to one project; member can have many tasks/logs	Lookup fields
Task	Task_ID, Task_Title, Project, Assigned_Member, Due_Date, Priority, Status	Project; Team Member; Contribution Log	One project/member to many tasks/logs	Lookup fields

4.3 Form Design

Table 4.2: Field Specification – Project Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Project ID	Project_ID	Auto Number	System generated	Prefix PRJ- if supported/configured	Unique project identifier
Project Name	Project_Name	Single Line	Yes	Trim spaces; sensible length	Display name of the project
Description	Description	Multi Line	No	—	Project scope and goals
Start Date	Start_Date	Date	Yes	Should not be after due date	Project start date
Due Date	Due_Date	Date	Yes	Should be on or after start date	Project deadline

4.4 Process and Workflow Design

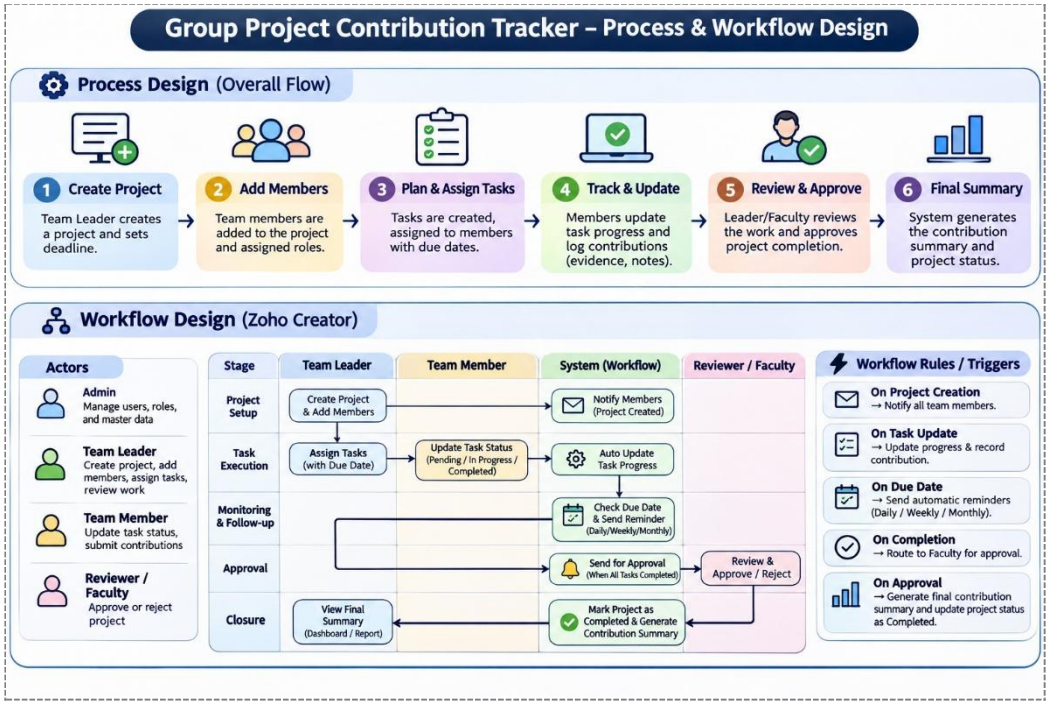


Figure 4.3: Contribution Workflow

4.5 User Interface and Navigation Design

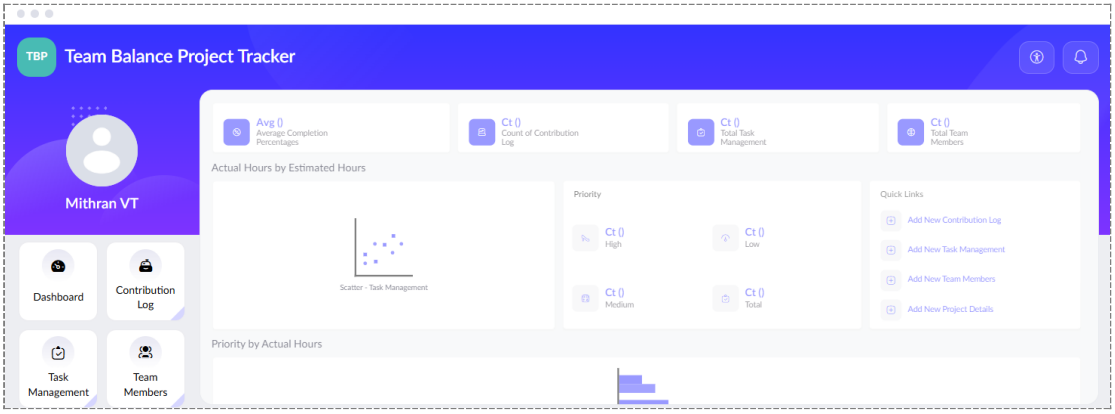


Figure 4.4: Navigation Structure and Dashboard Wireframe

4.6 Access Control Design

Table 4.3: Role–Permission Matrix

Component	Administrator	Coordinator	Student Member	Faculty Reviewer
Form: Project	A E V D	A E V	V	V
Form: Team Member	A E V D	A E V	V (relevant team)	V
Form: Task	A E V D	A E V	V; update own task if configured	V
Form: Contribution Log	A E V D	V E	A V E (own, if supported)	V
Report: Contribution Summary	V	V	V (own/relevant only, if configured)	V

CHAPTER 5
IMPLEMENTATION

5.1 Application Setup

Create an application in Zoho Creator named Group_Project_Contribution_Tracker (or the exact link name used in the live workspace). Organise the navigation into Projects, Team Members, Tasks, Contribution Logs, Reports and Dashboard. Use a consistent title and simple labels so that students can enter updates without training. Build the initial version with native Creator forms and reports, then add Deluge rules and permissions as required. Replace this description with the exact creation method and edition used in the final application.

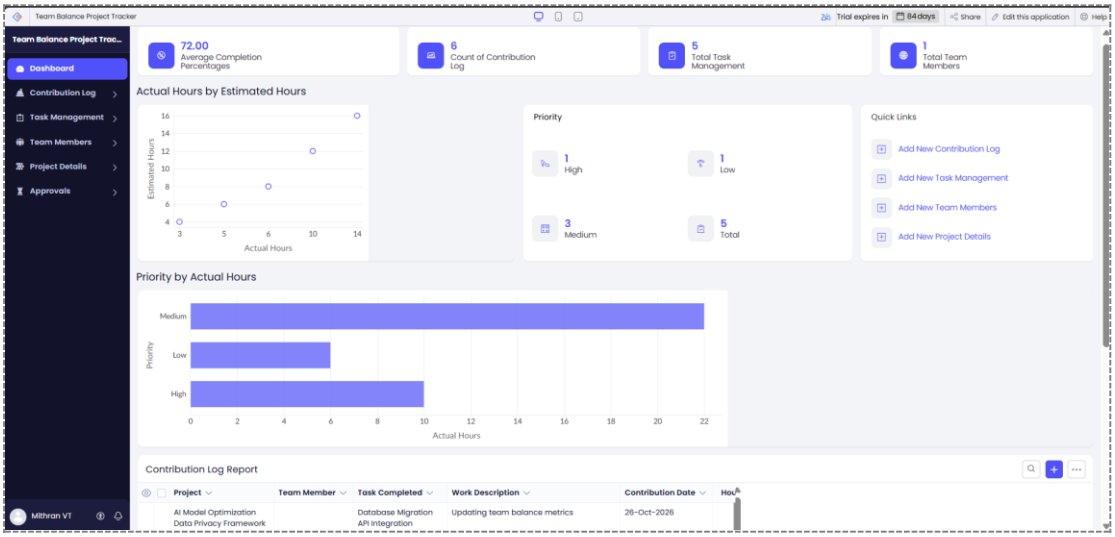


Figure 5.1: Group Project Contribution Tracker Home and Navigation

5.2 Forms and Data Capture

The Project form stores the project title, course or subject, description, start date, due date and overall status. The Team Member form stores the member's name, email, role and associated project. The Task form links each task to a project and an assigned member and stores its description, priority, due date and status. The Contribution Log form links to the relevant project, task and member and records the contribution date, description, effort hours and optional evidence. Use lookup fields to relate forms, mandatory fields for essential values, dropdowns for status and priority, and date/number validation where supported. The final report should list the exact fields and field types present in the live application.

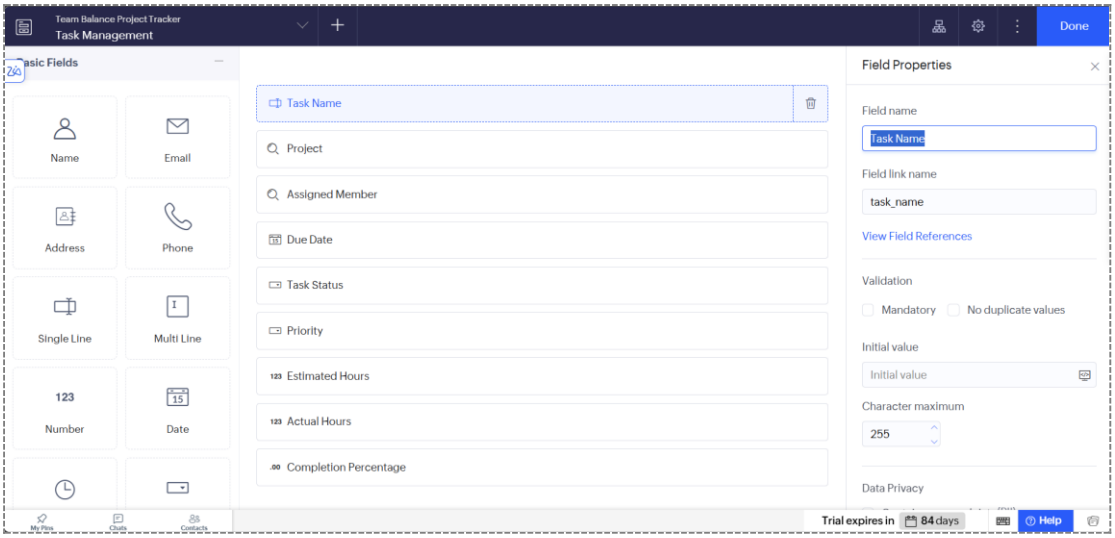


Figure 5.2: Task Form in the Form Builder

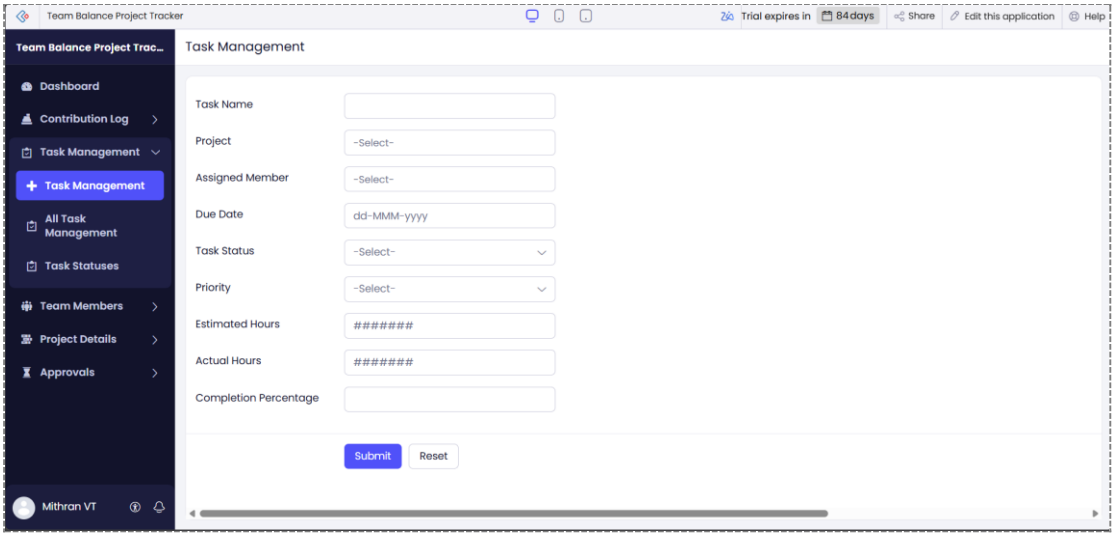


Figure 5.3: Task Form in Live Mode

5.3 Reports

Table 5.1: Reports Implemented

Report Name	Type	Source Form	Filter / Grouping / Sorting	Custom Actions	Roles
All Tasks	List	Task	Group by Status; sort by Due_Date ascending	Open/edit permitted record	Administrator, Coordinator
Task Pipeline	Kanban, if created	Task	Columns by Status	Move status only if configured	Coordinator, Student Member

Report Name	Type	Source Form	Filter / Grouping / Sorting	Custom Actions	Roles
Upcoming Deadlines	Calendar, if created	Task	Show tasks by Due_Date	View task details	Coordinator, Student Member
Contribution Summary	Pivot / Chart, if created	Contribution Log	Group by Project and Member; summarise effort hours if recorded	None assumed	Coordinator, Faculty Reviewer
My Contributions	List	Contribution Log	Filter to current member if record-level criteria configured	Add contribution	Student Member

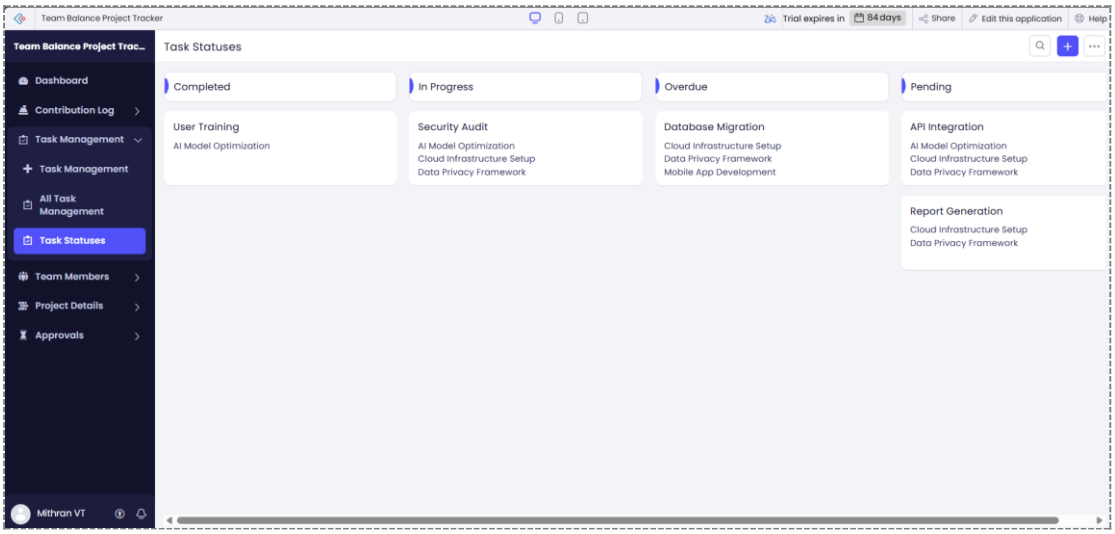


Figure 5.4: Task Status Report

5.4 Pages and Dashboards

The Dashboard page should provide a quick overview for a coordinator: total projects, open tasks, completed tasks, overdue tasks and recent contribution entries. A task report can be grouped by status and sorted by due date. A contribution report can be grouped by project and member. Charts should be based on saved records and should not imply

that self-reported effort hours are an objective measure of quality. Add only the page components and charts that are actually present in the application.

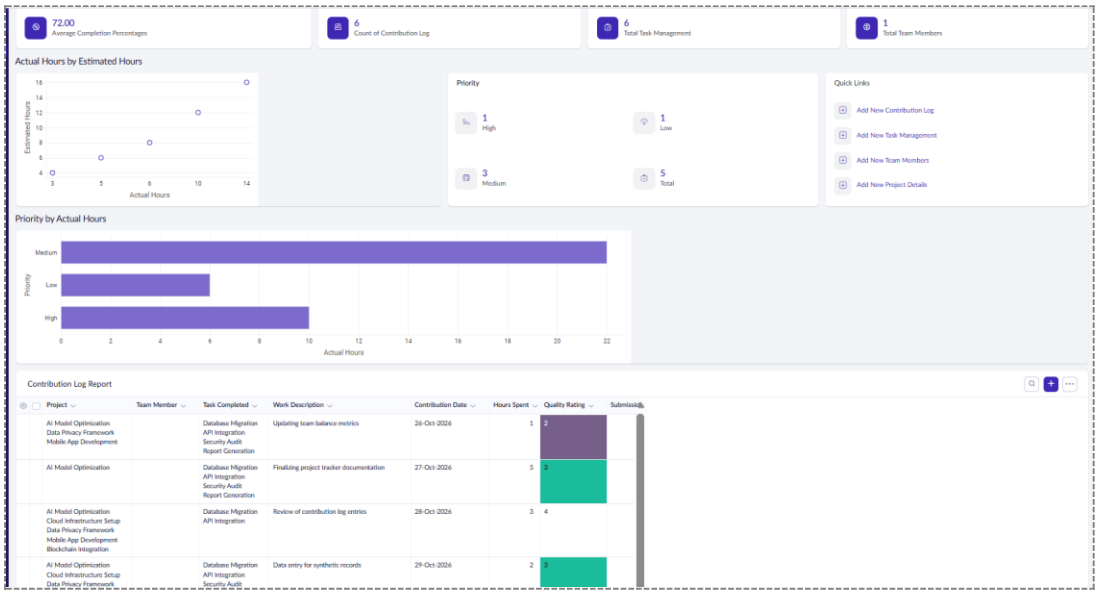


Figure 5.5: Project Overview Dashboard

5.5 Deluge Scripting

Table 5.2: Deluge Scripts Implemented

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-01	Task form — On Validate, if implemented	Check due date and required task details before save	if/else, validation and cancel submit
DS-02	Contribution Log form — On Validate, if implemented	Reject negative effort hours and incomplete contribution details	if/else, numeric comparison
DS-03	Contribution Log — On Success, if implemented	Notify coordinator about a new contribution	sendmail/workflow notification, string concatenation
DS-04	Scheduled workflow, if implemented	Send reminders for tasks approaching their due date	date criteria, scheduled workflow
DS-05	Dashboard/report configuration	Summarise task and contribution data	Report grouping and aggregation

Code 5.1: Example Validation Rule — replace with tested code from the live application

```
// DS-01 | Form: Order | Event: On Validate
// Purpose: Block the order if any item exceeds available stock
for each item in input.Order_Items
{
    prod = Product[ID == item.Product];
    if(item.Quantity > prod.Stock_Available)
    {
        alert "Insufficient stock for " + prod.Product_Name;
        cancel submit;
    }
}
```

Code 5.2: Example Notification Workflow — include only if implemented

```
// DS-02 | Form: Order | Event: On Success
sendmail
[
    from :zoho.adminuserid
    to :input.Manager_Email
    subject :"New order " + input.Order_ID + " awaiting approval"
    message :"Order placed by " + input.Customer.Customer_Name
]
```

The validation logic should check that required fields are completed, due dates are sensible and effort hours are non-negative. The notification workflow, if configured, should inform the coordinator when a new contribution is submitted or a task status changes. Test each script using valid and invalid records and inspect the execution or workflow logs. Replace the sample code and outcome statements with the exact scripts and observed results from the live application; do not report a script as implemented merely because it appears in this template.

5.6 Workflows and Automation

Table 5.3: Workflows Implemented

Workflow Name	Type	Trigger / Event	Condition	Action(s)	Script Ref.
Task Validation	Form Workflow	On Validate	Invalid or missing required task data	Prevent invalid submission	DS-01 if implemented
Contribution Validation	Form Workflow	On Validate	Effort hours below zero or required details missing	Show validation message; stop save	DS-02 if implemented

Workflow Name	Type	Trigger / Event	Condition	Action(s)	Script Ref.
Contribution Notification	Form Workflow	On Success	New contribution record created	Notify coordinator if configured	DS-03 if implemented
Deadline Reminder	Schedule	Daily at configured time	Task due date approaching and status not completed	Send reminder if configured	DS-04 if implemented
Task Lifecycle	Blueprint, optional	Status transition	Allowed transition from Planned to In Progress to Completed	Control status transitions if configured	Record actual script/workflow

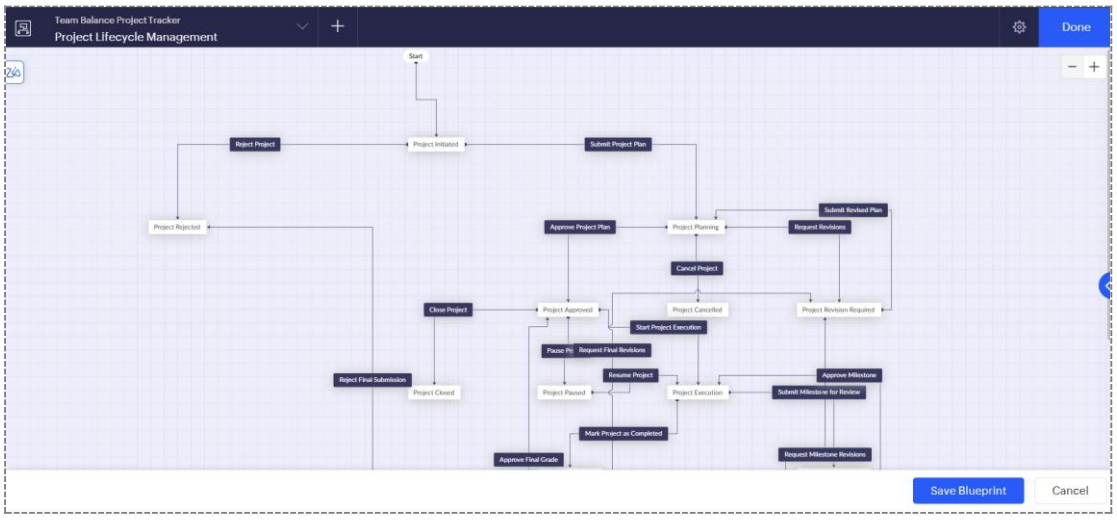


Figure 5.6: Workflow or Blueprint Configuration

5.7 Integrations

Table 5.4: Integrations Implemented

Integration	Service / Endpoint	Method	Authentication	Data Exchanged / Purpose
Zoho service notification (optional)	Zoho Mail / Creator notification	Built-in workflow action	Creator account permissions	Notify coordinator or member of task/contribution changes, if configured
External API	Not included in the assumed first version	N/A	N/A	No external API is assumed; add details only if implemented
Webhook / Creator REST API	Not included in the assumed first version	N/A	N/A	Potential future extension for external project-management tools

Code 5.3: External Integration — not applicable unless an API is configured

```
// External REST call using a secured connection
response = invokeurl
[
    url :"https://api.example.com/v1/resource?id=" + input.Ref_ID
    type :GET
    connection :"[connection_link_name]"
];
info response;
input.Result_Field = response.get("[key]");
```

5.8 User Management and Role-Based Access

Configure the application owner as administrator. If the project uses multiple Creator users, define a coordinator role that can create projects, assign tasks and review all contribution logs. Student members should be able to view relevant project and task details and create or update their own contribution entries. Use report criteria or record-level permissions where supported to limit access to appropriate records. Test each role by signing in with a separate test account. If the application is only a single-user prototype, state that limitation rather than claiming that role-based access has been deployed.

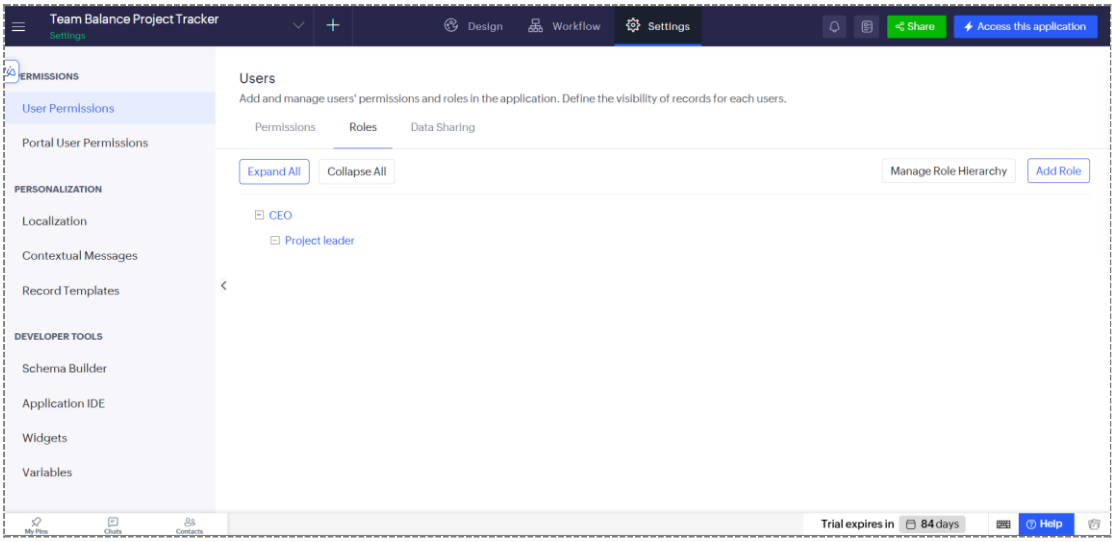


Figure 5.8: Roles and Permission Profiles Configuration

5.9 AI and Analytics Features

Use reports and dashboard charts to summarise task status, deadlines and contribution entries. If Zia or an AI-assisted builder was used to create any part of the application, identify the specific feature and explain how its output was checked. If no AI feature is implemented in the running application, state that analytics are limited to native reports and that AI-assisted analysis is a future enhancement.

5.10 Pro-Code Extension

No separate pro-code extension is included in the proposed first version. Zoho Creator forms, reports, pages and Deluge are sufficient for the basic tracking workflow. A future version could use Creator REST APIs, a custom widget or Zoho Catalyst if advanced visualisations, external integrations or processing beyond the low-code platform are required. This section should be updated if a pro-code component is actually built.

CHAPTER 6
TESTING AND DEBUGGING

6.1 Testing Strategy

Testing should cover individual forms, lookup relationships, field validation, reports, workflows and permissions. Positive tests should confirm that valid projects, members, tasks and contribution logs can be saved and displayed in reports. Negative tests should check missing mandatory values, invalid effort hours, invalid dates and access attempts by users without permission. Workflow and integration tests should be performed only for configured features. Record the actual result and pass/fail status for every test after executing it in Zoho Creator.

6.2 Test Cases

Table 6.1: Test Cases

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-01	FR-01	Create project with valid values	Enter project name, start date, due date and status; submit	Project record is saved	Record after running test	Pending
TC-02	FR-01, FR-06	Submit project without project name	Leave project name blank and submit	Required-field validation appears	Record after running test	Pending
TC-03	FR-01, FR-06	Enter due date before start date	Choose an earlier due date and submit	Invalid date order is prevented if validation configured	Record after running test	Pending
TC-04	FR-02	Register a team member	Enter name, email, role and project	Member record is saved and linked to project	Record after running test	Pending

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-05	FR-02	Submit member without required email	Leave email blank and submit	Validation appears if email is mandatory	Record after running test	Pending

6.3 Debugging Deluge Scripts

Use the Deluge editor's validation messages and debug output, workflow execution history and info statements to isolate script problems. Test one rule at a time using a small set of records. A suitable defect example to check is a lookup field returning a record ID while the script expects a display value; verify the actual field link name and lookup syntax, then correct the expression. Include this example in the final report only if it matches a bug actually encountered. Insert a screenshot of the real debug output.

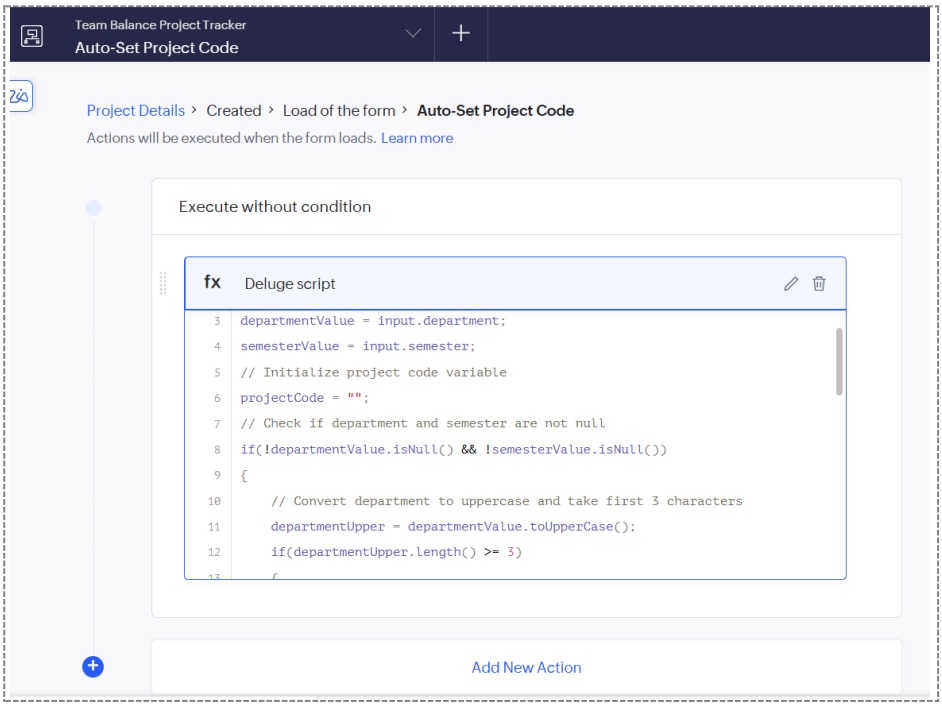


Figure 6.1: Deluge Debug Output or Workflow Log

6.4 Defect Log

Table 6.2: Defect Log

Bug ID	Description	Root Cause	Fix Applied	Status
BUG-01	Example to replace with an actual issue encountered	Determine from the real error or debug log	Document the actual correction after testing	Update after testing
BUG-02	No confirmed defect recorded in this draft	Not assessed yet	Run the application tests and document findings	Pending

6.5 User Acceptance Testing

Table 6.3: User Acceptance Feedback

User	Role Tested	Feedback	Action Taken
Test user 1 — enter name after testing	Student Member	Record actual feedback about task updates and contribution entry	Document any change made
Test user 2 — enter name after testing	Coordinator / Reviewer	Record actual feedback about reports and project overview	Document any change made

CHAPTER 7
DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

For a student prototype, first build and test in the development application. If the selected Zoho Creator edition provides separate environments, use a staging environment to verify forms, reports, workflows and permissions before publishing to production. Record the version or release notes and retain a rollback copy where the platform supports it. If separate environments are unavailable or not used, document the actual publish process honestly rather than claiming a development-stage-production pipeline.

Table 7.1: Deployment Checklist

S.No.	Checklist Item	Done (✓)
1	All test cases executed in Development / Stage	To verify
2	Sample/test data removed or isolated	To verify
3	Roles, permissions and users configured in Production	To verify
4	Connections authorised in Production	N/A unless integration used
5	Schedules and workflows enabled	To verify if workflows are used
6	Application published and URL verified	To verify
7	Version / release notes recorded	To verify

7.2 Performance Optimisation

Table 7.2: Performance Optimisation Results

Optimisation Applied	Component	Before	After	Improvement
[Replaced fetch inside loop with single criteria fetch]	[DS-03]	[ ]	[ ]	[ ]
[Filtered report by default criteria]	[All Orders report]	[ ]	[ ]	[ ]

7.3 Security Features

Table 7.3: Security Controls Implemented

Security Control	Where Applied	Threat Mitigated
Role/profile permissions	Creator users and permission profiles, if configured	Unauthorised access to forms and reports
Record-level criteria	Contribution Log / Task reports, if configured	Access to another member's records
Secure connection / OAuth	Only if external integration is configured	Credential leakage
Input validation	Project, Task and Contribution Log forms	Invalid or inconsistent data

7.4 Governance and Administration

Use clear link names for forms and fields, maintain a short change log, periodically export or back up application data where the edition permits, and review user access when team membership changes. Keep API keys and credentials out of scripts and use authorised connections for integrations. Review audit or activity history where available. The application owner should be responsible for checking workflows, removing test data and maintaining the user list.

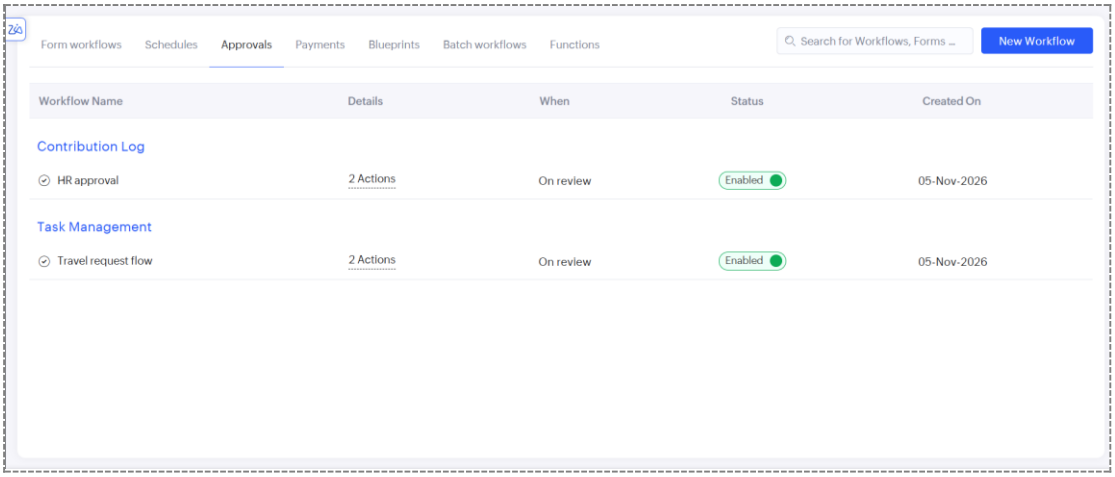


Figure 7.2: Audit Trail or Administration Settings

CHAPTER 8
RESULTS AND DISCUSSION

8.1 Application Output

Demonstration scenario: (1) the coordinator creates a project and registers team members; (2) a task is assigned to a member with a due date and priority; (3) the member records completed work in the Contribution Log and the dashboard/report reflects the new entry. Capture screenshots of these three stages from the working application and confirm that the records appear in the expected reports.

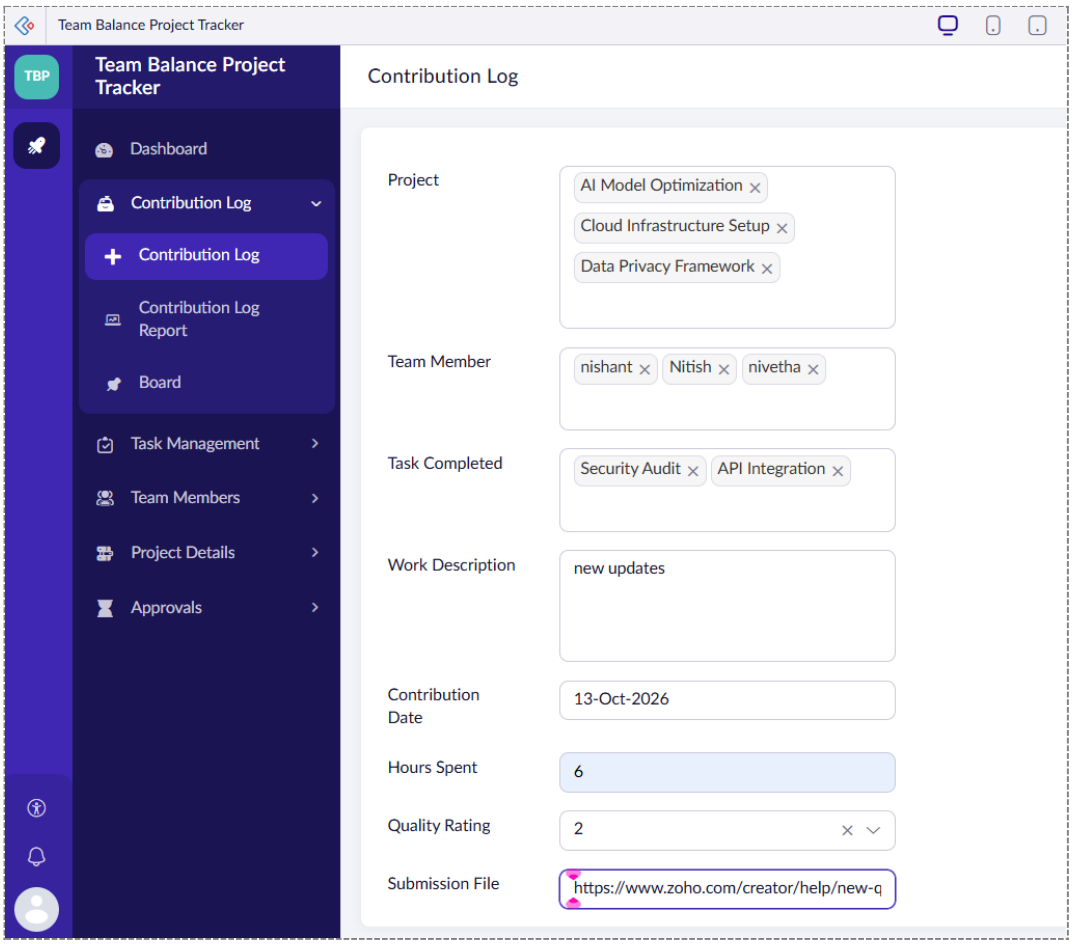


Figure 8.1: Step 1 — Coordinator Creates a Project

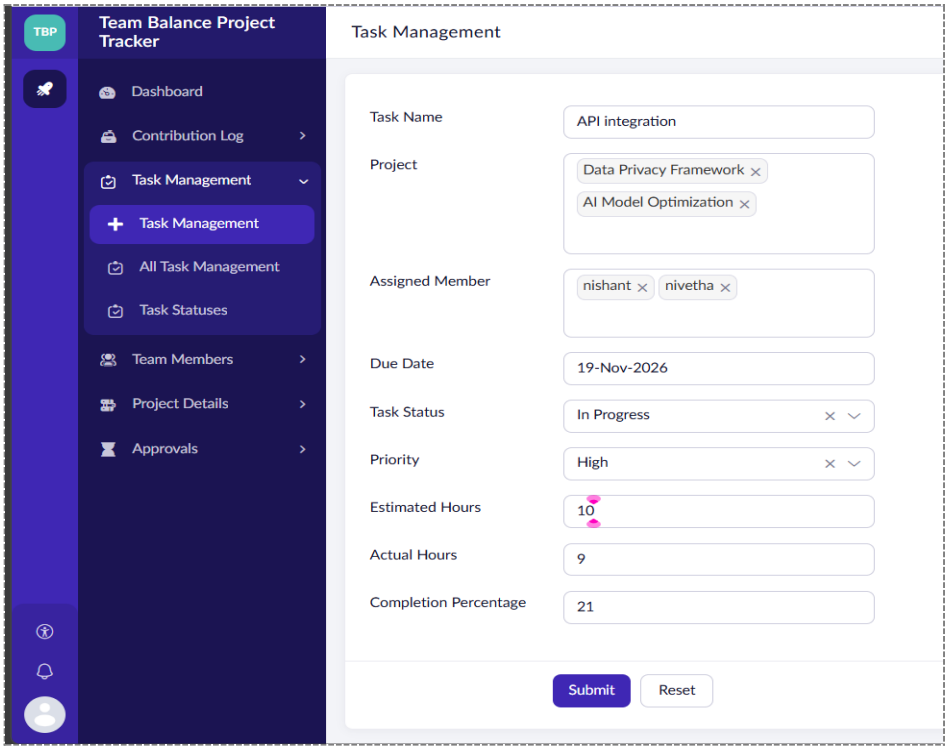


Figure 8.2: Step 2 — Coordinator Assigns a Task

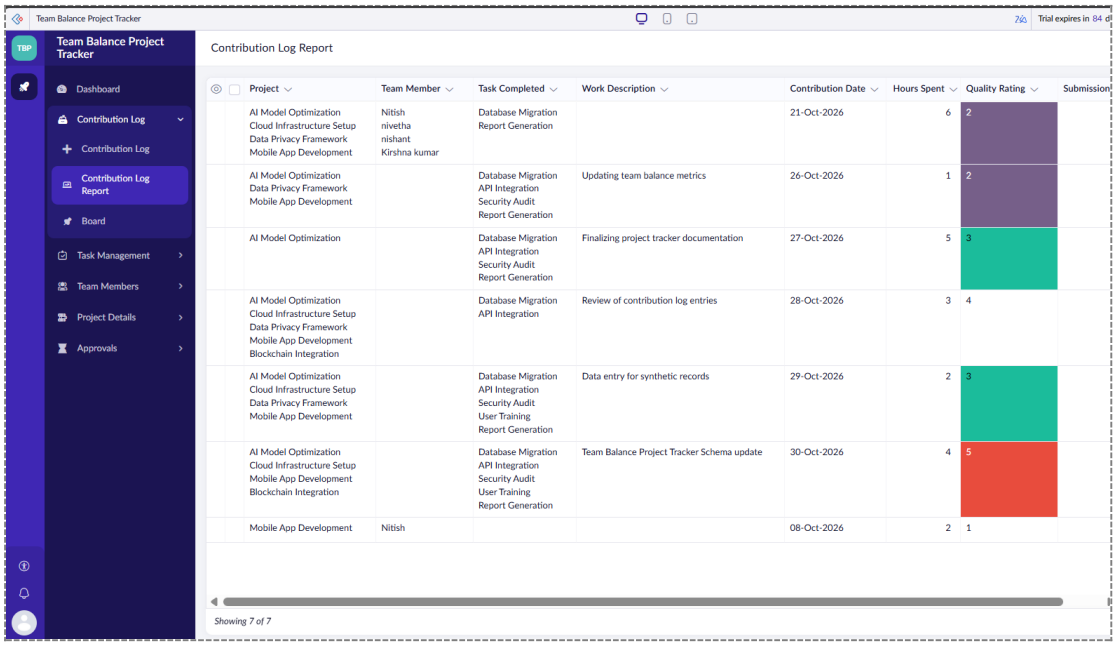


Figure 8.3: Step 3 — Member Records a Contribution

8.2 Achievement of Objectives

Table 8.1: Mapping of Objectives to Outcomes

Objective (Sec. 1.3)	Implemented Through	Outcome / Evidence
Structured data model and forms	Sections 4.2, 4.3 and 5.2	Evidence: form screenshots and linked sample records after implementation

Objective (Sec. 1.3)	Implemented Through	Outcome / Evidence
Automate validation and contribution updates	Sections 5.5 and 5.6	Evidence: actual Deluge/workflow configuration and test results
Provide reports and dashboard	Sections 5.3 and 5.4	Evidence: report and dashboard screenshots

8.3 Comparison with the Existing System

Table 8.2: Existing System versus Proposed Application

Parameter	Existing System	Proposed Application	Improvement
Time to process one update	Manual update across chat/spreadsheet	Single form submission and linked report	Measure with the same scenario before claiming a reduction
Manual steps	Messages, spreadsheet edits and manual summary	Structured project/task/contribution forms	Fewer scattered records; verify in demonstration
Report generation	Manual sorting and compilation	Filtered/grouped Creator reports	Report can use current saved data
Data accessibility	Depends on shared files and message history	Centralised access according to permissions	More consistent access, subject to configured permissions

8.4 Limitations of the Low-Code Approach

Zoho Creator's customisation options and Deluge language may not provide the same freedom as a fully custom application. Complex dashboards, specialised analytics or integrations may require additional configuration, API usage or a pro-code component. Platform capabilities and limits can also depend on the subscription edition. The initial design reduces complexity by using native forms, lookups, reports and straightforward workflows. Any limitation actually encountered should be described with the specific workaround used.

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

The Project, Team Member, Task and Contribution Log forms, basic reports and role configuration can largely be created through no-code visual configuration. Conditional validation, automated status changes and notifications require low-code Deluge or workflow rules. A future external service, custom interface or advanced analytics component may require pro-code development. The project demonstrates that the simplest platform feature should be preferred when it meets the requirement, while custom code should be introduced only when it adds necessary functionality.

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

The Group Project Contribution Tracker provides a structured approach to managing academic group work by linking projects, team members, tasks and contribution logs in one application. Its proposed reports help a coordinator review task progress, deadlines and recorded contributions without manually combining several spreadsheets. Zoho Creator is suitable for a first version because it offers visual form and report builders with workflow automation options. The final conclusion must be updated after implementation and testing to state which features were completed and what evidence was observed.

9.2 Future Enhancements

- Future enhancement 1: Add trend charts and optional Zia-assisted summaries to help coordinators review task progress, after validating the accuracy and availability of the feature.
- Future enhancement 2: Add automated email or in-app notifications for task assignments, approaching deadlines and contribution submissions, if these are not already implemented.
- Future enhancement 3: Add a faculty review module with peer feedback, evidence attachments and a secure API or custom service for advanced analytics if required.

9.3 Learning Outcomes and Career Relevance

This project provides practical experience in converting a coordination problem into a structured data model, building forms and reports, linking records, and planning workflow automation. It also develops an understanding of validation, access permissions, test design and deployment considerations. These skills are relevant to low-code developer, application support, business process automation and solutions consulting roles. The project also highlights the importance of testing generated or scripted logic rather than relying only on visual configuration.

REFERENCES

1. Zoho Corporation. *Zoho Creator – Low-Code Application Development Platform.* Available at: https://www.zoho.com/creator/
2. Zoho Corporation. *Understanding Forms in Zoho Creator.* Zoho Creator Help. Available at: https://help.zoho.com/portal/en/kb/creator/developer-guide/forms/the-concept-of-forms/articles/form-component
3. Zoho Corporation. *Understanding Reports in Zoho Creator.* Zoho Creator Help. Available at: https://help.zoho.com/portal/en/kb/creator/developer-guide/reports/understand-reports/articles/understand-reports
4. Zoho Corporation. *Understanding Workflows in Zoho Creator.* Zoho Creator Help. Available at: https://help.zoho.com/portal/en/kb/creator/developer-guide/workflows/understand-workflows/articles/understand-workflows
5. Zoho Corporation. *Creating and Managing Form Workflows.* Zoho Creator Help. Available at: https://help.zoho.com/portal/en/kb/creator/developer-guide/workflows/create-and-manage-form-workflows/articles/create-new-form-workflow
6. Zoho Corporation. *Understanding Approval Workflows.* Zoho Creator Help. Available at: https://help.zoho.com/portal/en/kb/creator/developer-guide/workflows/approval-workflows/articles/understanding-approval-workflows
7. Zoho Corporation. *Creating and Managing Approval Workflows.* Zoho Creator Help. Available at: https://help.zoho.com/portal/en/kb/creator/developer-guide/workflows/approval-workflows/articles/create-approval-workflow
8. Zoho Corporation. *AI Capabilities in Creator – Powered by Zia.* Zoho Creator Help. Available at: https://help.zoho.com/portal/en/kb/creator/developer-guide/operations/zia/articles/ai-capabilities-in-creator-powered-by-zia
9. Zoho Corporation. *Understanding and Configuring Zia Features.* Zoho Creator Help. Available at: https://help.zoho.com/portal/en/kb/creator/developer-guide/operations/zia/articles/configuring-zia

APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST

Table A.1: Feature Coverage Checklist

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
I	Application created & sections/menu organised	To verify	Enter actual form/report/workflow name	5.1	Add real figure/code reference
II	Forms with ≥ 8 distinct field types	To verify	Enter actual form/report/workflow name	5.2	Add real figure/code reference
II	Lookup field (relationship between forms)	To verify	Enter actual form/report/workflow name	4.2, 5.2	Add real figure/code reference
II	Subform (parent–child data)	To verify	Enter actual form/report/workflow name	4.2, 5.2	Add real figure/code reference
II	Field validations / layout rules	To verify	Enter actual form/report/workflow name	5.2	Add real figure/code reference
II	List report with filter, grouping, custom action	To verify	Enter actual form/report/workflow name	5.3	Add real figure/code reference
II	Kanban / Calendar / Timeline / Map report	To verify	Enter actual form/report/workflow name	5.3	Add real figure/code reference
II	Pivot table / chart report	To verify	Enter actual form/report/workflow name	5.3	Add real figure/code reference
II	Page / dashboard built with UI Builder	To verify	Enter actual form/report/workflow name	5.4	Add real figure/code reference
II	Deluge – form events (validate / success / field actions)	To verify	Enter actual form/report/workflow name	5.5	Add real figure/code reference
II	Deluge – functions, fetch, loops, collections	To verify	Enter actual form/report/workflow name	5.5	Add real figure/code reference

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
II	Email / in-app notifications	To verify	Enter actual form/report/workflow name	5.5, 5.6	Add real figure/code reference
II	Form workflow	To verify	Enter actual form/report/workflow name	5.6	Add real figure/code reference
II	Scheduled workflow	To verify	Enter actual form/report/workflow name	5.6	Add real figure/code reference
II	Approval process	To verify	Enter actual form/report/workflow name	5.6	Add real figure/code reference
II	Blueprint	To verify	Enter actual form/report/workflow name	5.6	Add real figure/code reference
II	Batch / report workflow	To verify	Enter actual form/report/workflow name	5.6	Add real figure/code reference
II	Testing & debugging evidence	To verify	Enter actual form/report/workflow name	6	Add real figure/code reference
III	Integration with a Zoho service	To verify	Enter actual form/report/workflow name	5.7	Add real figure/code reference
III	Connection / REST API call (invokeurl)	To verify	Enter actual form/report/workflow name	5.7	Add real figure/code reference
III	Webhook or Creator REST API	To verify	Enter actual form/report/workflow name	5.7	Add real figure/code reference
III	Users, roles and permission profiles	To verify	Enter actual form/report/workflow name	5.8	Add real figure/code reference
III	Record-level access criteria / customer portal	To verify	Enter actual form/report/workflow name	5.8	Add real figure/code reference
III	Zia / AI feature	To verify	Enter actual form/report/workflow name	5.9	Add real figure/code reference

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
III	Analytics (charts / Zoho Analytics)	To verify	Enter actual form/report/workflow name	5.9	Add real figure/code reference
IV	Performance optimisation applied & measured	To verify	Enter actual form/report/workflow name	7.2	Add real figure/code reference
IV	Deployment across environments	To verify	Enter actual form/report/workflow name	7.1	Add real figure/code reference
IV	Security controls (encryption, OAuth, permissions)	To verify	Enter actual form/report/workflow name	7.3	Add real figure/code reference
IV	Governance (audit trail, administration)	To verify	Enter actual form/report/workflow name	7.4	Add real figure/code reference
V	Pro-code extension (Catalyst / widgets / custom functions)	To verify	Enter actual form/report/workflow name	5.10	Add real figure/code reference
V	Low-code limitations & spectrum analysis	To verify	Enter actual form/report/workflow name	8.4, 8.5	Add real figure/code reference
—	Mobile app / portal access	To verify	Enter actual form/report/workflow name	5.11	Add real figure/code reference

Total features implemented: 32/ 32

APPENDIX B – COURSE OUTCOME MAPPING

Table B.1: Mapping of Course Outcomes to Report Chapters

CO	Course Outcome Statement	Evidenced In	Self-Assessed Level	Examiner Level
CO1	Explain the principles, architecture, and business value of low-code application development using Zoho Creator.	Ch. 1, 2	2 — Medium (revise based on work completed)	
CO2	Develop business applications using Zoho Creator forms, reports, workflows, and Deluge scripting.	Ch. 4, 5 (5.1–5.6), 6	2 — Medium (revise based on work completed)	
CO3	Integrate Zoho Creator applications with Zoho services and external systems using APIs, connections, and role-based security.	Ch. 4.6, 5.7–5.9	1 — Low unless integration is implemented	
CO4	Deploy, secure, and manage Zoho Creator applications across development and production environments following governance best practices.	Ch. 7	2 — Medium if deployment/security steps are completed	
CO5	Evaluate low-code development strategies, AI-assisted application development, and the transition from citizen developer to professional developer.	Ch. 5.9, 5.10, 8.4, 8.5, 9.3	2 — Medium	

APPENDIX C – COMPLETE DELUGE SOURCE CODE

C.1 Execute project code

```
// Auto-generate project code based on department and semester on form load
// Get department and semester values from input
departmentValue = input.department;
semesterValue = input.semester;
// Initialize project code variable
projectCode = "";
// Check if department and semester are not null
if(!departmentValue.isNull() && !semesterValue.isNull())
{
	// Convert department to uppercase and take first 3 characters
	departmentUpper = departmentValue.toUpperCase();
	if(departmentUpper.length() >= 3)
	{
		deptCode = departmentUpper.substring(0,3);
	}
	else
	{
		deptCode = departmentUpper;
	}
	// Convert semester to text and remove any non-numeric characters
	semesterText = semesterValue.toString();
	semesterClean = semesterText.replaceAll("[^0-9]","");
	// If semester is empty after cleaning, use default value
	if(semesterClean.isEmpty())
	{
		semesterClean = "01";
	}
	// Ensure semester code is at least 2 digits
	if(semesterClean.length() < 2)
	{
		semesterClean = "0" + semesterClean;
	}
	// Combine department code and semester to form project code
	projectCode = deptCode.concat("-").concat(semesterClean);
	// Set the project_code field value
	input.project_code = projectCode;
}
```

C.2 Notify team on task update

```
// Fetch the current task record being edited
if(input.task_status != "")
{
	// Get the assigned member lookup field
	assignedMemberLookup = input.assigned_member;
	// Check if assigned member is set
	if(assignedMemberLookup != null)
	{
		// Fetch the assigned member record to get email
		assignedMemberRecord = team_members[ID ==
assignedMemberLookup];
		// Get the email from the assigned member
		memberEmail = assignedMemberRecord.email;
		// Get task details
		taskName = input.task_name;
		newStatus = input.task_status;
		// Send notification email to the assigned member
		sendmail
		[
			from :zoho.loginuserid
			to :memberEmail
			subject :"Task Status Update: " + taskName
			message :"<p>Dear Team Member,</p><p>Your task
<strong>" + taskName + "</strong> has been updated to status: <strong>" +
newStatus + "</strong>.</p><p>Please review the changes and take necessary
action.</p>"
		]
	}
}
```

APPENDIX D – USER MANUAL

D.1 Accessing the Application

The Group Project Contribution Tracker is developed using Zoho Creator. Users can access the application through a web browser or a supported mobile interface, depending on the application's deployment and permissions.

Steps to access the application:

1. Open a web browser and navigate to the Zoho Creator application link provided by the administrator.

2. Log in using the authorized Zoho account or the access method configured for the application.

3. Open the Group Project Contribution Tracker application.

4. The home page or dashboard displays the available modules, such as Projects, Team Members, Tasks, Contributions, Reports and Approvals.

5. Select the required module to view, add or update information according to your permissions.

On a supported mobile device, open the application through the configured Zoho Creator mobile access method and sign in with the authorized account.

D.2 Instructions for [Role 1]

The administrator manages application access and oversees the overall operation of the system.

- **Step 1:** Log in to the application using an authorized administrator account.
- **Step 2:** Open the user and role management section to manage user access, where configured.
- **Step 3:** View and manage project records and related information.
- **Step 4:** Check reports and dashboards to monitor project and task progress.
- **Step 5:** Verify that the configured workflows, permissions and application settings operate correctly.

D.3 Instructions for [Role 2]

The team leader creates projects, adds members, assigns tasks and monitors the progress of the group.

- **Step 1:** Log in and open the Projects module.
- **Step 2:** Create a new project by entering its name, description, start date and deadline.
- **Step 3:** Open the Team Members module and add the students participating in the project.
- **Step 4:** Open the Tasks module and create tasks for the project.
- **Step 5:** Assign each task to a team member and specify its due date.
- **Step 6:** Review task statuses and contribution logs to monitor the work completed.
- **Step 7:** Open the Reports or Dashboard section to review project progress and contribution summaries.

D.4 Instructions for Team Member [Role 3]

Team members use the application to view assigned tasks, update their progress and record their contributions.

- **Step 1:** Log in using the authorized account.
- **Step 2:** Open the Tasks module and identify the tasks assigned to you.
- **Step 3:** Review the task description and due date.
- **Step 4:** Update the task status to Pending, In Progress or Completed, according to the available options and the actual progress.
- **Step 5:** Open the Contributions module and submit a contribution record.
- **Step 6:** Enter the task details, work description and contribution date.
- **Step 7:** Add an evidence link if the application provides that field.
- **Step 8:** Review the available reports to check your task and contribution records.

D.5 Instructions for Reviewer or Faculty [Role 4]

The reviewer or faculty member reviews project submissions and checks the group's progress, where approval features have been configured.

- **Step 1:** Log in using the authorized account.
- **Step 2:** Open the Approvals module.
- **Step 3:** Select the project submission that requires review.
- **Step 4:** Examine the project details, task statuses and contribution records.
- **Step 5:** Approve or reject the submission using the available options.
- **Step 6:** Enter review comments if required.
- **Step 7:** View the final project summary or related reports, if access is permitted.

APPENDIX E – EVALUATION RUBRIC

(For Examiner Use Only)

Student Name	MITHRAN VT	Register No.	240701312
Project Title	Group Project Contribution Tracker	Date	

Table E.1: Evaluation Rubric for the Individual Project

S.No.	Criterion	Indicators	Max. Marks	Marks Awarded
1	Problem Definition & Requirement Analysis	Clarity of problem, measurable objectives, complete FR/NFR, use cases	10	
2	System Design	Architecture, ER/data model, form specifications, workflow design, permission matrix	10	
3	Forms, Reports & Pages	Variety of field types, lookups/subforms, report types, UI Builder dashboards	15	
4	Deluge Scripting & Workflows	Correctness and depth of scripts; form, schedule, approval, Blueprint, batch workflows	15	
5	Integration, RBAC & AI	Zoho/external integrations, connections, roles & permissions, Zia/analytics	10	
6	Testing, Deployment, Security & Governance	Test coverage, environments, optimisation, security controls, audit	10	
7	Live Demonstration	End-to-end working of the application across roles; feature checklist verified	15	
8	Project Report	Adherence to template and format, clarity, figures, references	5	
9	Viva-Voce	Understanding of the platform, design decisions and low-code concepts	10	
	TOTAL		**100**	

Table E.2: Performance Level Descriptors

Level	% of Max. Marks	Descriptor
Excellent	90–100	Feature implemented completely, correctly and beyond the basic requirement; student explains design choices confidently.
Good	70–89	Feature implemented correctly with minor gaps; explanation adequate.
Satisfactory	50–69	Feature partially implemented or works only for the main path; limited explanation.
Needs Improvement	< 50	Feature missing, non-functional or not understood by the student.

Signature of Internal Examiner

Name: _____

Signature of External Examiner

Name: _____

APPENDIX F – VIVA-VOCE RECORD SHEET

(For Examiner Use Only)

Student Name	MITHRAN VT	Register No.	240701312
Department / Section	Computer Science and Engineering	Date of Viva	For examiner to complete
Application URL Verified	Yes / No	Demo Duration	_____ min

Table F.1: Viva-Voce Questions and Responses

Q.No.	Question / Topic Asked	Unit	Response (E / G / S / NI)	Remarks
1				
2				
3				
4				
5				
6				
7				
8				

E – Excellent G – Good S – Satisfactory NI – Needs Improvement

Table F.2: Marks Summary

Component	Max. Marks	Marks Awarded
Criteria 1–8 (Appendix E)	90	
Viva-Voce	10	
TOTAL	**100**	

Examiners' Remarks:

_____

_____

INTERNAL EXAMINER

Name: _____

EXTERNAL EXAMINER

Name: _____